L09: Python Essentials for Simulation

Simulation Without a Black Box

Outline

Learning Objectives

Generators and `yield`

Dataclasses for Parameters

Type Hints

Reproducible Randomness with NumPy

SciPy Distributions

Collecting Output with pandas

Key Takeaways

Learning Objectives

By the end of this lecture you will be able to:

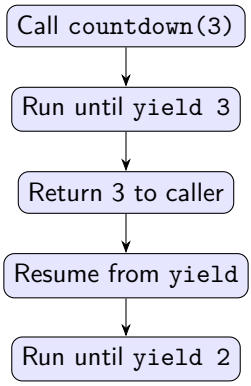
- Explain how Python generators underpin SimPy's event model.
- Use `dataclasses` to create clean, type-annotated parameter bundles.
- Generate independent, reproducible random streams with `numpy.default_rng`.
- Select the right SciPy distribution for a given inter-arrival or service pattern.
- Collect output across many replications into a tidy pandas DataFrame.

Generators and `yield`: The SimPy Foundation

A **generator function** returns a *generator object* that pauses at every `yield` and resumes when `next()` is called.

```
def countdown(n):
    while n > 0:
        yield n        # pause here
        n -= 1

gen = countdown(3)
next(gen)  # 3
next(gen)  # 2
next(gen)  # 1
```



Why it matters for SimPy

SimPy *is* a scheduler for generator objects. Each process is a generator; `yield env.timeout(t)` suspends the process for simulated time t.

Dataclasses for Parameter Bundles

Avoid scattered magic numbers. Bundle model parameters into a typed, inspectable object.

```python
from dataclasses import dataclass

@dataclass
class MM1Params:
    arrival_rate: float   # lambda
    service_rate: float   # mu
    sim_time:     float = 10_000.0
    seed:         int   = 42

p = MM1Params(arrival_rate=0.8,
              service_rate=1.0)
rho = p.arrival_rate / p.service_rate
```

Benefits

- Type hints are enforced by type checkers (`mypy`).
- `asdict(p)` serialises to JSON for experiment logs.
- `dataclasses.replace(p, seed=99)` creates a variant without mutating the original.
- IDEs auto-complete field names.

Type Hints on Public Functions

```python
import numpy as np
from numpy.random import Generator

def run_one_rep(params: MM1Params, rng: Generator) -> dict[str, float]:
    """Run one replication and return summary statistics.

    Args:
        params: Model parameters (arrival rate, service rate, run length).
        rng: NumPy random generator (caller-owned, seeded externally).

    Returns:
        Dictionary with keys 'mean_wait', 'utilisation', 'throughput'.
    """
    ...
```

- Annotate every public function and class method (PEP 484).
- Run `mypy simdes/simdes/` in CI — treat type errors as build failures.
- Use `float`, not `np.float64`, in signatures for interoperability.

NumPy default_rng: Reproducible Randomness

```python
import numpy as np

rng = np.random.default_rng(seed=42)

# Exponential inter-arrivals (rate=0.8)
iat = rng.exponential(scale=1/0.8)

# Spawn child streams (independent!)
n_reps = 30
child_rngs = rng.spawn(n_reps)

for i, child in enumerate(child_rngs):
    result = run_one_rep(params, child)
```

Key rules

1. Never use `np.random.seed()` — it sets global state.
2. Pass `rng` as a constructor argument.
3. Use `rng.spawn(n)` for n independent streams — avoids stream overlap.
4. Store the seed in the output for reproducibility.

Common mistake

Reusing the same `rng` across replications introduces correlation

SciPy Distributions for Service and Arrival Times

Distribution	SciPy name	CV	Typical use
Exponential	`expon`	1.0	Poisson arrivals, memoryless service
Erlang-k	`gamma(a=k)`	$1/\sqrt{k}$	Staged processing, low variability
Lognormal	`lognorm`	> 1 typical	Skewed service times (healthcare)
Weibull	`weibull_min`	shape-dependent	Failure times, bathtub hazard
Deterministic	constant	0.0	M/D/1 baseline

Sampling pattern (all distributions)

- Use `rng.exponential(scale=1/mu)` not `scipy.stats.expon.rvs()` in hot loops — NumPy is 5–20× faster.
- Use SciPy for `.cdf()`, `.ppf()`, and goodness-of-fit tests.

pandas for Multi-Replication Output

```
import pandas as pd

records = []
for i, child_rng in enumerate(rng.spawn(n_reps)):
    stats = run_one_rep(params, child_rng)
    records.append({"rep": i, **stats})

df = pd.DataFrame(records)
# df columns: rep | mean_wait | utilisation | throughput

summary = df[["mean_wait","utilisation"]].agg(["mean","sem"])
ci95 = summary.loc["mean"] + 1.96 * summary.loc["sem"] * [-1, 1]
```

- Collect one dict per replication; build the DataFrame *after* the loop.
- `df.to_csv("output/mm1_results.csv", index=False)` preserves runs.
- Never store the raw time-series inside the DataFrame — summarise first.

Key Takeaways

Core ideas from L09

1. **Generators + `yield`** are not just syntactic sugar — they are SimPy's entire concurrency model.
2. **Dataclasses** turn scattered constants into self-documenting, type-checked parameter objects.
3. **`default_rng` + `spawn`** gives independent, reproducible streams with no global side effects.
4. **SciPy** names the distribution; **NumPy** samples it fast in simulation loops.
5. **pandas** is the natural container for replication output — tidy format from the start.

Next: L10 — SimPy fundamentals and your first event-driven simulation.